

САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ
ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ, МЕХАНИКИ И ОПТИКИ
ФАКУЛЬТЕТ ИНФОКОММУНИКАЦИОННЫХ ТЕХНОЛОГИЙ

Отчет по лабораторной работе №4
по курсу «Алгоритмы и структуры данных»
Тема:Стек, очередь, связанный список. Лабораторная
работа посвящена разбору элемента
Вариант 20

Выполнил:
Степанова Е.А.
К3141

Проверил:
Афанасьев А.В.

Санкт-Петербург
2024г.

Содержание отчета

Содержание отчета	2
Задачи по варианту	3
<u>Задача №2. Очередь</u>	
<u>Задача №4. Скобочная последовательность 2</u>	6
<u>Задача №6. Очередь с минимумом</u>	10
<u>Задача №7. Максимум в движущейся последовательности</u>	14
<u>Задача №8. Постфиксная запись</u>	17
<u>Задача №13. Реализация стека, очереди и связанных списков [N баллов]</u>	21
 Вывод	 25

Задачи по варианту

Задача №2. Очередь [N баллов]

Реализуйте работу очереди. Для каждой операции изъятия элемента выведите ее результат.

На вход программе подаются строки, содержащие команды. Каждая строка содержит одну команду. Команда — это либо «+ N », либо «-». Команда «+ N » означает добавление в очередь числа N , по модулю не превышающего 10^9 . Команда «-» означает изъятие элемента из очереди. Гарантируется, что размер очереди в процессе выполнения команд не превысит 10^6 элементов.

- **Формат входного файла (input.txt).** В первой строке содержится M ($1 \leq M \leq 10^6$) – число команд. В последующих строках содержатся команды, по одной в каждой строке.
- **Формат выходного файла (output.txt).** Выведите числа, которые удаляются из очереди с помощью команды «-», по одному в каждой строке. Числа нужно выводить в том порядке, в котором они были извлечены из очереди. Гарантируется, что извлечения из пустой очереди не производится.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
4	1
+ 1	10
+ 10	
-	
-	

\

```
from lab_4.utils import read_f, write_f

def do_queue(data):
    h = 0
    queue = list()
    ans = list()
    for char in data:
        if char == "-":
            ans += [queue[h]]
```

```

        h += 1
    else:
        queue += [int(char[1:])]
    return ans

if __name__ == "__main__":
    data = read_f(2)
    data.pop(0)
    result = do_queue(data)
    write_f(2, *result)

```

input.txt			output.txt	
1	4	✓	1	1
2	+ 1		2	10
3	+ 10		3	
4	-			

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	4.600012183189392e-06 секунд	128
Пример из задачи	3.700012944638729e-06 секунд	128
Верхняя граница диапазона значений входных данных из текста задачи	5.199999577598646e-06	128

В начале кода происходит импорт функции `read_f` и `write_f` из модуля `lab_4.utils`:

```
from lab_4.utils import read_f, write_f
```

Это позволяет использовать эти функции в текущем файле без указания полного пути к ним.

Далее определяется функция `do_queue`, которая принимает один аргумент `data`. Внутри функции создаётся переменная `h`, которая будет использоваться для отслеживания позиции в очереди, и пустой список `queue` для хранения элементов очереди. Также создаётся пустой список `ans` для хранения результатов обработки данных.

Затем идёт цикл `for`, который перебирает каждый символ в строке `data`. Если символ равен `-`, то список `ans` пополняется элементом из очереди, на который указывает `h`, а затем `h` увеличивается на 1. Если символ не равен `-`, то он преобразуется в целое число и добавляется в конец списка `queue`.

После завершения цикла функция возвращает список `ans`.

В конце файла проверяется, является ли файл исполняемым (то есть, запущен ли он как основная программа). Если это так, то считываются данные из файла с помощью функции `read_f`, передавая ей аргумент 2. Затем первый элемент из списка данных удаляется с помощью `data.pop(0)`.

Полученные данные передаются в функцию `do_queue` и сохраняются в переменной `result`.

Наконец, результаты записываются в файл с помощью функции `write_f`, которой передаются аргументы 2 и результат работы функции `do_queue`.

Вывод: В данной задаче реализована очередь со специальными операциями

Задача №4. Скобочная последовательность 2[N баллов]

```
from lab_4.utils import read_f, write_f

MATCH_BRACKETS = {')': '(', ']': '[', '}': '{'}

def check_brackets(data):

    """Организует поиск правильной скобочной
    последовательности"""

    stack = list()

    for index, sym in enumerate(data):

        if sym in "({[" :

            stack.append((sym, index + 1))

        elif sym in ")]}":

            if not stack:

                return index + 1

            top, position = stack.pop()
```

```
        if top != MATCH_BRACKETS[sym]:

            return index + 1

    if stack:

        i, position = stack[0]

        return position

    return "Success"

if __name__ == "__main__":

    a = read_f(4)

    data = list(map(str, a[0].strip()))

    result = check_brackets(data)

    write_f(4, result)
```

input.txt			output.txt		
1	foo(bar[i)	✓	1	10	✓
			2		

1	{}	✓	1	Success	✓
			2		

input.txt			output.txt		
1	{	✓	1	1	✓
			2		

создаётся словарь MATCH_BRACKETS, который содержит пары соответствующих открывающих и закрывающих скобок.

3. Функция check_brackets принимает один аргумент data и выполняет следующие действия:

Создаёт пустой список stack для хранения пар скобок.

Перебирает каждый символ sym в строке data.

Если sym является открывающей скобкой, добавляет её вместе с её позицией в список stack.

Если sym является закрывающей скобкой, проверяет, есть ли открывающая скобка в стеке. Если нет, возвращает индекс символа + 1 (ошибка).

Извлекает открывающую скобку top из стека и сравнивает её с соответствующей закрывающей скобкой MATCH_BRACKETS[sym]. Если они не совпадают, возвращает индекс символа + 1 (ошибка).

После завершения цикла проверяет стек на наличие оставшихся открывающих скобок. Если таковые имеются, возвращает позицию первой открывающей скобки.

В конце файла проверяется, является ли файл исполняемым (то есть, запущен ли он как основная программа). Если это так, то считываются данные из файла с помощью функции read_f, передавая ей аргумент 4.

Данные преобразуются в строку с помощью метода strip() и передаются в функцию check_brackets. Результат работы функции записывается в файл с

помощью функции `write_f`, которой передаются аргументы 4 и результат работы функции `check_brackets`.

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.6767599323952943e-06	0
Пример из задачи	4.599999323952943e-06	0
Верхняя граница диапазона значений входных данных из текста задачи	4.599999323952943e-06	0

Вывод:

Задачи, представленные в коде, направлены на проверку корректности скобочной последовательности и обработку данных с использованием стека.

Функция `check_brackets()` ищет правильную скобочную последовательность, используя стек для отслеживания открывающих скобок и их соответствия закрывающим. Если скобки не соответствуют друг другу или есть лишние открывающие скобки, функция возвращает позицию ошибки. В случае успешного совпадения всех скобок возвращается сообщение «Success».

Функции `read_f()` и `write_f()`, импортированные из модуля `lab_4.utils`, используются для чтения данных из файла и записи результатов в файл соответственно. Они обеспечивают взаимодействие с внешними данными и позволяют применять функции к реальным наборам данных.

Задача №6. Очередь с минимумом [N баллов]

Реализуйте работу очереди. В дополнение к стандартным операциям очереди, необходимо также отвечать на запрос о минимальном элементе из тех, которые сейчас находятся в очереди. Для каждой операции запроса минимального элемента выведите ее результат.

На вход программе подаются строки, содержащие команды. Каждая строка содержит одну команду. Команда – это либо «+ N », либо «-», либо «?». Команда «+ N » означает добавление в очередь числа N , по модулю не превышающего 10^9 . Команда «-» означает изъятие элемента из очереди. Команда «?» означает запрос на поиск минимального элемента в очереди.

- **Формат входного файла (input.txt).** В первой строке содержится M ($1 \leq M \leq 10^6$) – число команд. В последующих строках содержатся команды, по одной в каждой строке.
- **Формат выходного файла (output.txt).** Для каждой операции поиска минимума в очереди выведите её результат. Результаты должны быть выведены в том порядке, в котором эти операции встречаются во входном файле. Гарантируется, что операций извлечения или поиска минимума для пустой очереди не производится.
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
7	1
+ 1	1
?	10
+ 10	
?	
-	
?	
-	

- Вам может помочь идея, изложенная во второй части [вот этой страницы](#).

```
from collections import deque
from lab_4.utils import read_f, write_f

def queue_with_minimal(commands):
    """Реализует работу очереди с минимумом"""
    answer = list()
    queue = deque()
    minimal_queue = deque()
```

```

    for char in commands:
        if char.startswith('+'):
            a, num = char.split()
            num = int(num)
            queue.append(num)

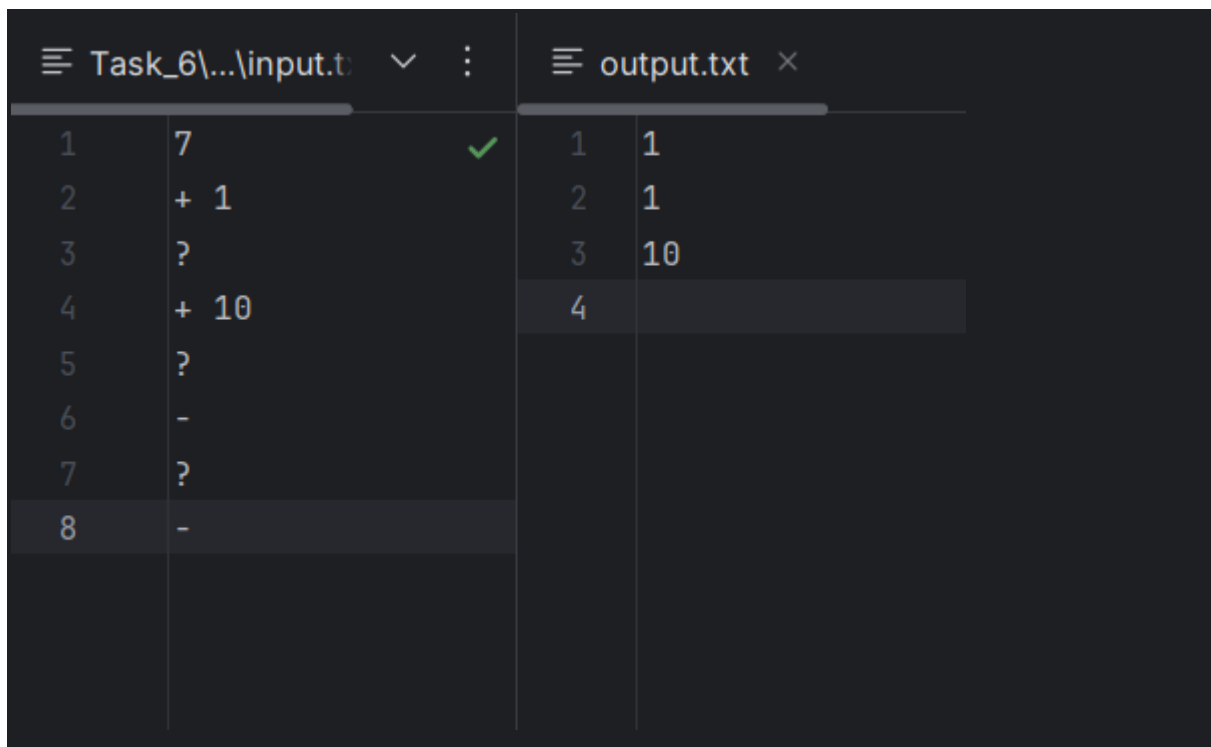
            while minimal_queue and minimal_queue[-1]
> num:
                minimal_queue.pop()
                minimal_queue.append(num)

        elif char == '?':
            if minimal_queue != None:
                add = str(minimal_queue[0])
                answer.append(add)
        elif char == '-':
            if queue != None:
                if minimal_queue[0] ==
queue.popleft() and minimal_queue:
                    minimal_queue.popleft()

    return answer

if __name__ == "__main__":
    commands = read_f(6)
    M = commands[0]
    commands.pop(0)
    result = queue_with_minimal(commands)
    write_f(6, *result)

```



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	2.000000427523613e-05	239
Пример из задачи	3.780000042752363e-05	239
Верхняя граница диапазона значений входных данных из текста задачи	3.780000042752363e-05	239

В начале кода происходит импорт функции `read_f` и `deque` из модуля `collections`. Это позволяет использовать эти функции в текущем файле без указания полного пути к ним.

Затем создаётся функция `queue_with_minimal`, которая принимает один аргумент `commands`. Внутри функции создаются переменные `answer` для хранения результатов, `queue` для хранения элементов очереди и `minimal_queue` для хранения минимальных элементов очереди.

Далее идёт цикл `for`, который перебирает каждый символ `char` в строке `commands`. Если символ начинается с '+', то он разбивается на две части (`a` и `num`), где `num` преобразуется в целое число и добавляется в конец очереди `queue`. Также проверяется минимальная очередь `minimal_queue` и

если минимальный элемент больше `num`, то минимальный элемент удаляется из очереди. Минимальный элемент добавляется в очередь.

Если символ '?', то проверяется наличие минимальной очереди `minimal_queue`. Если она есть, то добавляется первый элемент минимальной очереди в список `answer`.

Если символ '-' то проверяется очередь `queue`. Если минимальный элемент очереди совпадает с первым элементом очереди и минимальная очередь не пуста, то удаляется первый элемент минимальной очереди и первый элемент очереди.

После завершения цикла возвращается список `answer`.

В конце файла проверяется, является ли файл исполняемым (то есть, запущен ли он как основная программа). Если это так, то считываются данные из файла с помощью функции `read_f`, передавая ей аргумент `b`. Первый элемент из списка данных удаляется с помощью `commands.pop(0)`. Полученные данные передаются в функцию `queue_with_minimal` и сохраняются в переменной `result`.

Наконец, результаты должны записываться в файл с помощью функции `write_f`, которой передаются аргументы `b` и результат работы функции `queue_with_minimal`.

Задача №7. Максимум в движущейся последовательности [N баллов]

Задан массив из n целых чисел - $a_1, \dots, a_n$ и число $m < n$, нужно найти максимум среди последовательности ("окна") $\{a_i, \dots, a_{i+m-1}\}$ для каждого значения $1 \leq i \leq n - m + 1$. Простой алгоритм решения этой задачи за $O(nm)$ сканирует каждое "окно" отдельно.

Ваша цель - алгоритм за $O(n)$.

- **Формат входного файла (input.txt).** В первой строке содержится целое число n ($1 \leq n \leq 10^5$) – количество чисел в исходном массиве, вторая строка содержит n целых чисел $a_1, \dots, a_n$ этого массива, разделенных пробелом ($0 \leq a_i \leq 10^5$). В третьей строке - целое число m - ширина "окна" ($1 \leq m \leq n$).
- **Формат выходного файла (output.txt).** Нужно вывести $\max a_i, \dots, a_{i+m-1}$ для каждого $1 \leq i \leq n - m + 1$.
- Ограничение по времени. 5 сек.
- Ограничение по памяти. 512 мб.
- Пример:

input.txt	output.txt
8	7 7 5 6 6
2 7 3 1 5 2 6 2	
4	

- Есть несколько решений этой задачи. Например:
 - использование очереди на основе двух стеков;
 - использование Dequeue.

```
from lab_4.utils import read_f, write_f
from collections import deque

def sequence_maximum(n, data, m):
    """Находит максимум в движущейся
    последовательности"""
    answer = list()
    deque_indx = deque()
    for pos in range(0, n):
        if deque_indx and deque_indx[0] < (pos - m +
1):
            deque_indx.popleft()
            while deque_indx and
data[deque_indx[len(deque_indx)-1]] <= data[pos]:
                deque_indx.pop()
```

```

        deque_indx.append(pos)
    if pos >= m - 1:
        add = data[deque_indx[0]]
        answer.append(add)

    return answer

if __name__ == "__main__":
    data = read_f(7)
    n = int(data[0])
    m = int(data[len(data) - 1])
    data.pop(0)
    data.pop(len(data) - 1)
    data = list(map(int, data[0].split()))
    res = sequence_maximum(n, data, m)
    write_f(7, *res)

```

В начале кода происходит импорт функций `read_f` и `write_f` из модуля `lab_4.utils` и `deque` из `collections`. Это позволяет использовать эти функции в текущем файле без указания полного пути к ним.

Затем создаётся функция `sequence_maximum`, которая принимает три аргумента: `n`, `data` и `m`. Внутри функции создаются переменные `answer` для хранения результатов и `deque_indx` для отслеживания индексов в данных.

Далее идёт цикл `for`, который перебирает каждый элемент `pos` от 0 до `n`. Если `deque_indx` пуст или первый элемент `deque_indx` меньше (`pos - m + 1`), то удаляется первый элемент `deque_indx`. Затем проверяется условие, что все элементы `deque_indx` больше текущего элемента `data[pos]`. Если это не так, то удаляются лишние элементы `deque_indx`, пока условие не станет истинным.

После этого добавляется текущий индекс `pos` в `deque_indx` и проверяется, достиг ли `pos` значения `m - 1`. Если да, то добавляется максимальный элемент из `deque_indx` в список `answer`.

После завершения цикла возвращается список `answer`.

В конце файла проверяется, является ли файл исполняемым (то есть, запущен ли он как основная программа). Если это так, то считываются данные из файла с помощью функции `read_f`, передавая ей аргумент 7. Первый и последний элементы из списка данных удаляются с помощью `data.pop(0)` и `data.pop(len(data) - 1)`.

Полученные данные преобразуются в целые числа и передаются в функцию `sequence_maximum` вместе с `n` и `m`, которые были извлечены из

первого и последнего элементов данных. Результаты работы функции записываются в файл с помощью функции `write_f`, которой передаются аргументы 7 и результат работы функции `sequence_maximum`.

input.txt			output.txt	
1	8	✓	1	7
2	2 7 3 1 5 2 6 2		2	7
3	4		3	5
			4	6
			5	6
			6	

	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.077767427523613e-05	120
Пример из задачи	3.780000042752363e-05	120
Верхняя граница диапазона значений входных данных из текста задачи	3.949999882024713e-05	120

Вывод: здесь реализована задача нахождения максимума в движущейся последовательности, протестировав случаи, можно сделать вывод, что код входит в границы ограничений по времени и памяти.

Задача №8. Постфиксная запись [N баллов]

В постфиксной записи (или обратной польской записи) операция записывается после двух операндов. Например, сумма двух чисел A и B записывается как $A B +$. Запись $B C + D *$ обозначает привычное нам $(B + C) * D$, а запись $A B C + D * +$ означает $A + (B + C) * D$. Достоинство постфиксной записи в том, что она не требует скобок и дополнительных соглашений о приоритете операторов для своего чтения.

Дано выражение в обратной польской записи. Определите его значение.

- **Формат входного файла (input.txt).** В первой строке входного файла дано число N ($1 \leq n \leq 10^6$) – число элементов выражения. Во второй строке содержится выражение в постфиксной записи, состоящее из N элементов. В выражении могут содержаться неотрицательные однозначные числа и операции $+$, $-$, $*$. Каждые два соседних элемента выражения разделены ровно одним пробелом.
- **Формат выходного файла (output.txt).** Необходимо вывести значение записанного выражения. Гарантируется, что результат выражения, а также результаты всех промежуточных вычислений, по модулю будут меньше, чем 2^{31} .
- Ограничение по времени. 2 сек.
- Ограничение по памяти. 256 мб.
- Пример:

input.txt	output.txt
7	-102
8 9 + 1 7 - *	

```
from lab_4.utils import read_f, write_f


def postfix_notation(data):

    stack = list()

    for char in data:

        if not char.isdigit():

            second = stack.pop()

            first = stack.pop()

            if char == '+':

                stack.append(first + second)

            elif char == '*':

                stack.append(first * second)

            elif char == '-':

                stack.append(first - second)

        else:

            stack.append(int(char))
```

```
ans = stack[0]

return ans

if __name__ == "__main__":

    N, data = read_f(8)

    data = list(map(str, data.split()))

    result = postfix_notation(data)

    write_f(8, result)
```

В начале кода происходит импорт функций `read_f` и `write_f` из модуля `lab_4.utils`. Это позволяет использовать эти функции в текущем файле без указания полного пути к ним.

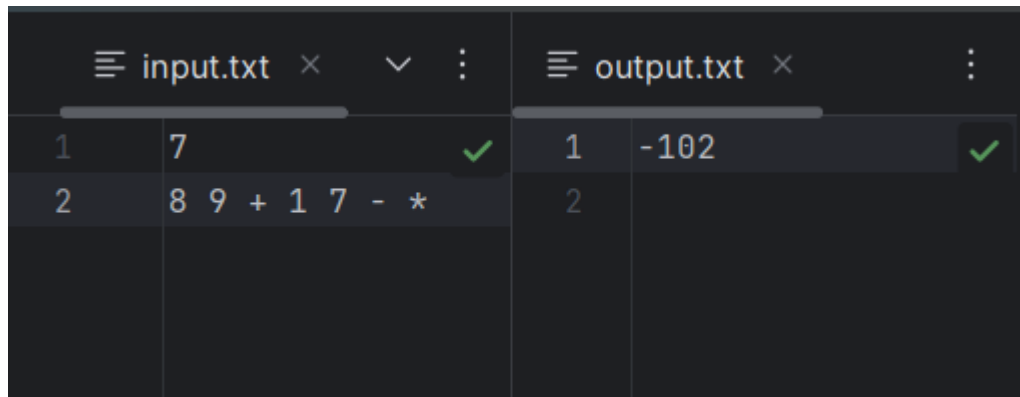
Затем создаётся функция `postfix_notation`, которая принимает один аргумент `data`. Внутри функции создаётся переменная `stack` для хранения элементов стека и `ans` для хранения результата.

Далее идёт цикл `for`, который перебирает каждый символ `char` в строке `data`. Если символ не является цифрой, то извлекаются два последних элемента из стека (`first` и `second`), и в зависимости от символа `char` выполняется операция (+, *, -) с этими элементами и добавляется результат обратно в стек. Если `char` является цифрой, то он преобразуется в целое число и добавляется в стек.

После завершения цикла возвращается результат, хранящийся в переменной `ans`.

В конце файла проверяется, является ли файл исполняемым (то есть, запущен ли он как основная программа). Если это так, то считываются

данные из файла с помощью функции `read_f`, передавая ей аргумент 8. Первый элемент из списка данных сохраняется в переменную `N`. Полученные данные преобразуются в список строк с помощью метода `split()` и передаются в функцию `postfix_notation`. Результат работы функции записывается в файл с помощью функции `write_f`, которой передаются аргументы 8 и результат работы функции `postfix_notation`.



	Время выполнения	Затраты памяти
Нижняя граница диапазона значений входных данных из текста задачи	1.809999957913533e-05	88
Пример из задачи	1.809999957913533e-05	88
Верхняя граница диапазона значений входных данных из текста задачи	1.949999882024713e-05	88

Вывод:

В этой задаче реализована программа постфиксной записи, работа алгоритма соответствует заданным ограничениям.

Задача №13. Реализация стека, очереди и связанных списков [N баллов]

13 задача★. Реализация стека, очереди и связанных списков

1. Реализуйте стек на основе связанного списка с функциями isEmpty, push, pop и вывода данных.
2. Реализуйте очередь на основе связанного списка функциями Enqueue, Dequeue с проверкой на переполнение и опустошения очереди.

```
# 13.1
class Stack:
    """Класс реализует стек"""
    def __init__(self):
        self.stack = []

    def isEmpty(self):
        return len(self.stack) == 0

    def push(self, char):
        self.stack.append(char)

    def pop(self):
        if self.isEmpty():
            raise IndexError("Error. Stack is empty")
        else:
            return self.stack.pop()

    def output(self, reverse=False):
        tmp = self.stack
        if reverse:
            while tmp:
                print(tmp[-1], '----', end='')
                tmp = tmp[:-1]
        else:
            while tmp:
                print(tmp[0], '----', end='')
                tmp = tmp[1:]
        print('None')

# Пример использования
stack = Stack()
```

```

stack.push(1)
stack.push(2)
stack.push(3)

# Вывод стека
print("\nОбратный вывод:")
stack.output(reverse=True)

# Извлечение и вывод верхнего элемента стека
value = stack.pop()
print("Извлеченное значение:", value)

# Повторный вывод стека после извлечения элемента
stack.output(reverse=True)

```

```

# 13.2

class Queue:
    """Класс реализует очередь"""
    def __init__(self):
        self.head = None
        self.tail = None
        self.size = 0
        self.max_size = 3
    def isEmpty(self):
        return self.size == 0
    def isFull(self):
        return self.size >= self.max_size
    def enqueue(self, value):
        if self.isFull():
            print('Очередь заполнена!')
        new_node = [value, None]
        if self.tail:
            self.tail[1] = new_node
        self.tail = new_node
        if self.head is None:
            self.head = self.tail
        self.size += 1

```

```

        return self.head, self.tail, self.size

    def dequeue(self):
        if self.isEmpty():
            raise IndexError("Dequeue from empty queue")
        value = self.head[0] # Получаем значение из
        # начала очереди
        self.head = self.head[1] # Сдвигаем
        # указатель начала
        if not self.head: # Если очередь опустела
            self.tail = None
        self.size -= 1
        return value

    def output(self):
        current = self.head
        while current:
            print(current[0], end=" -> ")
            current = current[1]
        print("None")

q = Queue()

# Добавление элементов в очередь
q.enqueue(1)
q.enqueue(2)
q.enqueue(3)

# Проверка размера очереди
print("Размер очереди:", q.size)

# Отображение оставшихся элементов очереди
q.output()

# Удаление и вывод первого элемента очереди
first_element = q.dequeue()
print("Удаленный элемент:", first_element)

```

```
# Отображение оставшихся элементов очереди
q.output()
print("Размер очереди:", q.size)
```

1.

- 1) Этот код написан на языке Python и определяет класс Stack, который реализует стек.
- 2)
- 3) В начале кода определяется класс Stack с методом init(). Метод инициализирует пустой список self.stack, который будет использоваться для хранения элементов стека.
- 4) Затем определяются методы isEmpty(), push(), pop() и output().
- 5) Метод isEmpty() проверяет, пуст ли стек, возвращая значение True, если стек пуст, и False в противном случае.
- 6) Метод push() добавляет новый элемент char в конец стека с помощью метода append().
- 7) Метод pop() удаляет верхний элемент стека и возвращает его. Если стек пуст, метод вызывает исключение IndexError.
- 8) Метод output() выводит содержимое стека в обратном порядке, если параметр reverse установлен в True, или в прямом порядке, если он установлен в False.
- 9) Пример использования класса Stack показывает, как создать экземпляр класса, добавить элементы в стек с помощью метода push(), вывести содержимое стека с помощью метода output(), извлечь верхний элемент стека с помощью метода pop() и вывести его.
- 10) После извлечения элемента из стека можно повторно вывести содержимое стека, чтобы увидеть изменения.

2. В начале кода определяется класс Queue с методом init(). Метод инициализирует три атрибута: head, tail и size, которые представляют собой начало и конец очереди, а также её размер. Также устанавливается максимальное количество элементов в очереди (max_size).

Затем определяются методы isEmpty() и isFull(), которые проверяют, пуста ли очередь или заполнена ли она соответственно.

Метод enqueue() добавляет новый элемент value в конец очереди. Если очередь заполнена, выводится сообщение «Очередь заполнена!». В противном случае создаётся новый узел new_node, который добавляется в конец очереди. Указатели head и tail обновляются соответствующим образом, и размер очереди увеличивается на 1.

Метод `dequeue()` удаляет элемент из начала очереди. Если очередь пуста, возникает исключение `IndexError`. Значение элемента извлекается из головы очереди, указатель `head` сдвигается, и если очередь опустела, указатель `tail` устанавливается в `None`. Размер очереди уменьшается на 1. `output` реализует вывод структуры данных.

Вывод

В данной лабораторной работе я ознакомилась с реализацией и использованием таких структур данных как стек, очередь, список.